

South Emerald Supermarket Inventory - Project Documentation

Autor: South Emerald Inventory System | Data: 01/10/2026

South Emerald Supermarket Inventory — Project Documentation

Generated: 2026-10-01

Repository: project11 (package name: inventory-system, version 0.1.0)

1. Overview

South Emerald Supermarket Inventory is a web-based inventory and point-of-sale system built for a single grocery-store location in Tiaong, Quezon. It helps store owners, managers, inventory personnel, and cashiers register products, identify them through barcodes, keep stock quantities accurate through receiving and customer purchases, monitor stock and expiry status, and produce inventory and gross-profit reports.

The system joins camera-based or hardware barcode identification directly to simple stock-movement workflows, including generated Code 128 labels for grocery items that do not already carry manufacturer barcodes.

Primary users: grocery-store owners, managers, inventory staff, cashiers.

Operating context: desktop and laptop computers with built-in or external webcams, mobile devices with cameras, and USB barcode scanners that behave as keyboard input. Staff often work quickly near shelves, receiving areas, and checkout counters.

Product principles

- Make stock movement immediately understandable.
- Optimize scanning and quantity updates for speed and low error rates.
- Keep inventory status visible and traceable.
- Keep development seed records separate from production business data.
- Work comfortably across desktop, webcam, and mobile-camera contexts.

Explicitly out of scope

Supplier management, delivery tracking, multi-location management, advanced warehouse management, full accounting, and payment processing.

2. Technology Stack

Layer	Technology
Framework	Next.js 16.3.5 (App Router)
UI	React 19.3.0, TypeScript 5.9
Styling	Tailwind CSS 4.3 (PostCSS), custom CSS variables
Components	shadcn/ui-style components on Radix UI primitives (dialog, tabs, label, separator)
Icons	@phosphor-icons/react
Charts	Chart.js 4.5 via react-chartjs-2
Barcodes	jsbarcode 3.12 (Code 128 rendering), @zxing/browser 0.2 (camera decoding)
Animation	motion 13.2
Validation	Zod 4.6
Backend	Supabase — PostgreSQL, Auth, Row-Level Security, PostgREST RPCs
Data cache	IndexedDB (latest catalog and activity snapshot)
Testing	Vitest 5 (unit tests, jsdom)
Linting	ESLint 9 with eslint-config-next

3. Architecture

Request surfaces

The application exposes Next.js Server Actions and Supabase RPC functions. It does not expose custom Route Handlers under `app/api`.

- **Server Actions** (`src/app/<domain>/actions.ts`) are the write and read entry points from the client. Every action is treated as a public entry point: authentication comes from verified Supabase claims plus an active-profile lookup, authorization is re-checked inside each action, and client values are parsed at runtime with Zod or explicit numeric checks.
- **Database RPC functions** repeat role and ownership checks so direct Supabase callers cannot bypass application authorization.
- **Next.js** provides same-origin checks for Server Actions and a default 1 MB action-body limit.

Application layout

- `src/app/page.tsx` — the authenticated single-page shell; sidebar selections switch views while the browser stays at /
- `src/app/login` — the separate sign-in route
- `src/components` — view components (dashboard, products, sales, returns, stock-in, reports, history, verification, staff accounts)
- `src/lib` — domain logic: cart, pagination, units, barcode values, stock alerts, reporting, inventory cache, validation, permissions, rate limiting

- src/hooks — barcode scanner capture hook
- supabase/migrations — the source of truth for the database schema

Data flow

1. On sign-in the app loads the inventory snapshot and opens the Dashboard.
2. The latest product catalog and activity are cached in IndexedDB to speed repeated lookup and tolerate transient failures while the app remains open.
3. Supabase remains the source of truth. Sale confirmation, stock receiving, and all writes require a live connection; the app never confirms a sale from cached-only or offline data.
4. Camera access requires user permission and a secure browser context (localhost or HTTPS).

4. Features and Views

Navigation labels are views inside the authenticated application, not separate URLs. The login screen is the /login route.

Dashboard

Quick overview of inventory health and recent activity. Summary metrics vary by role; charts show stock movement and inventory health; attention items identify low and out-of-stock products. Shortcuts lead into Sales, Receive Stock, and Activity.

Products

Search and manage the active catalog, categories, barcode labels, and archived products. Administrators and managers can create and edit products, assign categories and one operational unit of measure, assign manufacturer barcodes or generated INV-##### Code 128 labels, and archive with a reason. Archiving with remaining stock requires an acknowledgement and records a write-off for every affected batch. Administrators alone may permanently delete an archived product after confirming its product code. Active and archived tables paginate locally (10/25/50/100 rows, default 25).

Sales

A workspace combining product scanning and stock-out recording. Each successful scan adds one unit to the local receipt cart, repeated scans increment the same line, and quantities remain editable. A Price Check mode shows an active product's price, unit, description, and availability without touching the cart or database; it clears after 15 seconds. A configured scanner prefix (F9) plus Enter suffix enables scan-anywhere routing without corrupting focused fields. Cash checkout requires the amount received, displays change before confirmation, and validates payment against the database-calculated total. Supabase is updated only when the cashier confirms the complete receipt as one atomic transaction.

Completed sales deduct non-expired inventory using FEFO batch allocation with idempotency protection.

Returns

Returns stay linked to their original sale. Staff find a sale by number or cashier, choose return quantities and per-item condition (resellable, damaged, or expired), add a reason and notes, and submit for manager review. Damaged, expired, and rejected returns do not increase available stock. Approval happens in Daily Verification.

Receive Stock

Record incoming supplier batches with quantity, unit cost, batch details, manufacture/expiry dates, delivery reference, and notes. Scanning selects a product but never changes inventory by itself; switching products clears the previous draft, and confirmation is always mandatory.

Stock Levels

Filter across All, In Stock, Low Stock, and Out of Stock. Shows product, category, quantity, minimum level, status, and last update.

Activity and Receipts

Three tabs: Stock activity, Sales receipts, and Customer returns. Selecting a receipt opens full details, linked returns, and a chronological sale/return/verification timeline, with 80 mm printing or browser Save as PDF.

Daily Verification

Business days move from open to pending review to verified. A day is created automatically when the first sale is confirmed. Review a day's sales, resolve pending returns (pending returns block verification only for their own day), submit the day for review, then mark it verified.

Reports

Sales (by receipt or by item), stock movement, and expiring inventory reports with date ranges, CSV download, and print options where supported. Available reports depend on reporting permissions.

Staff Accounts

Read-only list of staff profiles, roles, and active status in the current interface.

Notifications

The header notification center derives low-stock and out-of-stock alerts from the latest inventory snapshot against each product's restock level, with direct routing into Stock In for authorized staff.

5. Roles and Permissions

Roles: administrator, manager, inventory_staff, cashier. Permissions are enforced both in the interface and again on the server.

Permission	Administrator	Manager	Inventory Staff	Cashier
---	::	::	::	::
dashboard:view	Yes	Yes	Yes	Yes
products:view	Yes	Yes	Yes	Yes
products:manage	Yes	Yes	No	No
products:archive	Yes	Yes	No	No
products:delete	Yes	No	No	No
stock:receive	Yes	Yes	Yes	No

sales:record	Yes	Yes	Yes	Yes	
sales:verify	Yes	Yes	No	No	
adjustments:manage	Yes	Yes	No	No	
transactions:view_own	Yes	Yes	Yes	Yes	
transactions:view_all	Yes	Yes	Yes	No	
reports:view_sales_own	Yes	Yes	Yes	Yes	
reports:view_sales_all	Yes	Yes	No	No	
reports:view_inventory	Yes	Yes	Yes	No	
reports:view_costs	Yes	Yes	No	No	
users:manage	Yes	No	No	No	

6. Database Schema

Migrations live in `supabase/migrations` and are the source of truth. `supabase/master.sql` is a one-shot installer for a new empty project (regenerate with `npm run db:master`).

Tables

Table	Purpose	
---	---	
profiles	Staff profiles and roles, created by trigger from auth.users	
categories	Product categories	
products	Product catalog, codes, units, restock levels, archive state	
product_barcodes	Manufacturer barcode and generated label aliases	
product_costs	Purchase cost history	
inventory_batches	Traceable receiving batches with expiry dates	
inventory_adjustments	Stock adjustments and write-offs	
batch_allocations	FEFO allocation of sale lines to batches	
sales	Permanent sale receipts	
sale_items	Receipt line items	
sale_item_costs	Cost snapshots for sold items	
sales_returns	Return requests linked to their original sale	
sales_return_items	Per-item quantities and conditions	
return_batch_allocations	Inventory restored when a return is approved	
business_days	Open / pending review / verified day records	
audit_logs	Audit trail	
endpoint_rate_limits	Shared, atomic rate-limit counters	

Key database functions

- `record_sale` — confirms a complete receipt in one transaction: writes receipt and lines, snapshots costs, allocates non-expired batches by FEFO, deducts inventory
- `get_inventory_catalog`, `get_inventory_activity`, `get_archived_inventory_products` — protected read models
- `get_recent_sales`, `get_sale_for_return` — sales lookup for the sales and returns flows
- `request_sale_return`, `review_sale_return`, `get_pending_return_summaries` — return workflow
- `get_business_day_summaries`, `get_business_day_sales`, `submit_business_day`, `verify_business_day` — day verification workflow
- `get_stock_movement_report`, `get_expiring_inventory_report` — reporting
- `consume_endpoint_rate_limit` — shared rate-limit counter
- `handle_new_user` — profile trigger; new users default to the cashier role so they cannot self-elevate

Public registration is disabled. The first administrator is promoted manually from the Supabase SQL editor (see section 9).

7. Server Action API Surface

| Domain file | Actions |

| --- | --- |

| `src/app/login/actions.ts` | `login`, `logout` |

| `src/app/inventory/actions.ts` | `listInventoryCategories`, `createInventoryCategory`, `updateInventoryCategory`, `createInventoryProduct`, `createInventoryProductWithInitialStock`, `updateInventoryProduct`, `listArchivedInventoryProducts`, `archiveInventoryProduct`, `restoreInventoryProduct`, `permanentlyDeleteArchivedProduct`, `receiveInventoryStock` |

| `src/app/sales/actions.ts` | `recordSale`, `findSaleForReturn`, `listRecentSales`, `requestSaleReturn`, `loadBusinessDaySales`, `loadSalesVerification`, `changeBusinessDayStatus`, `reviewSaleReturn` |

| `src/app/history/actions.ts` | `loadTransactionHistory` |

| `src/app/reports/actions.ts` | `loadSalesReport`, `loadStockMovementReport`, `loadExpiringInventoryReport` |

| `src/app/staff/actions.ts` | `createStaffAccount`, `updateStaffAccount` |

`logout` intentionally accepts an unauthenticated request because clearing an absent session is harmless.

8. Security

Verification rules

- Authentication from verified Supabase claims, then an active-profile lookup.
- Authorization checked again inside each action; hiding a control in the UI is not authorization.
- Client values parsed at runtime with Zod or explicit numeric checks.
- Database RPC functions repeat role and ownership checks for direct Supabase callers.
- Returned objects contain only the fields the interface requires.

Rate limits

Scope	Limit	Enforcement
---	---	---
Login, per account and address	5 per 15 minutes	Server Action + database counter
Login, per address	30 per 15 minutes	Server Action + database counter
Completed sales	30 per minute	Database trigger
Return requests	20 per hour	Database trigger
Inventory writes	60 per 10 minutes	Database trigger
Permanent deletion	10 per hour	Database trigger
Verification and return reviews	60 per hour	Database trigger
Reports	60 per minute	Server Action + database counter
Receipt/history views	60 per minute	Server Action + database counter
Sales lookups	120 per minute	Server Action + database counter
Inventory lookups	120 per minute	Server Action + database counter

Write limits run inside PostgreSQL triggers, so an authenticated caller cannot bypass them by calling a Supabase mutation RPC directly. Counters are atomic and shared across instances; raw email addresses and network addresses are not stored in the limiter table. The application fails closed when the shared limiter is unavailable.

Deployment requirements

1. Apply migrations through `20260921000800_endpoint_rate_limits.sql` before deploying the matching build.
2. Ensure the production reverse proxy replaces (does not append) X-Forwarded-For.
3. Configure the same `NEXT_SERVER_ACTIONS_ENCRYPTION_KEY` on every Next.js instance.
4. Monitor `RATE_LIMITED` database errors and repeated login failures; never log passwords, raw session tokens, or full payment details.
5. Keep Supabase Auth and PostgREST platform rate limits enabled as the outer layer.

9. Setup and Installation

Environment variables

Copy `.env.example` to `.env.local`:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_your_key
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key
```

The service-role key is server-only — never prefix it with `NEXT_PUBLIC_` and never commit it. Without it, role changes still work but adding an account does not.

Database setup

```
Copy-Item .env.example .env.local
npx supabase login
npx supabase link --project-ref YOUR_PROJECT_REF
npx supabase db push
```

For a brand-new empty project, paste `supabase/master.sql` into the Supabase SQL editor and run it once. It is transactional and stops if it detects an existing schema. Do not use it to upgrade an existing database — apply migrations in filename order instead.

Seed development data (synthetic, development-only) with `supabase/seed.sql`. Reset a development project with `supabase/reset-development-data.sql`; that preserves Auth users and profiles. Never run the seed or reset scripts against production.

First administrator

Public registration is disabled. Create the first user in Supabase Dashboard under Authentication > Users with a `full_name` in user metadata; the trigger creates a cashier profile. Then promote it:

```
update public.profiles
set role = 'administrator'
where id = (
select id from auth.users where email = 'replace-with-admin-email@example.com'
);
```

Scripts

Command	Purpose
---	---
<code>npm run dev</code>	Start the development server
<code>npm run build</code>	Production build
<code>npm run start</code>	Start the production server
<code>npm run lint</code>	ESLint
<code>npm run test</code>	Run the Vitest suite once
<code>npm run test:watch</code>	Vitest in watch mode
<code>npm run db:master</code>	Regenerate <code>supabase/master.sql</code> from migrations
<code>npm run db:master:check</code>	Verify the master installer matches the migrations

Local verification

Docker is required for the local Supabase stack.

```
npx supabase start
npx supabase db reset
```

```
npx supabase test db
npm run test
npm run lint
npm run build
```

Authentication settings

- Disable public email sign-ups in hosted Supabase Auth.
- Use administrator-created accounts only.
- Set the Site URL and allowed redirect URLs for local, preview, and production.
- Configure production SMTP before enabling password recovery.
- Keep development and production credentials separate.

10. Design System

Defined in design-system/inventory-system/MASTER.md. Dials: Variance 5/10 (Balanced), Motion 3/10 (Subtle), Density 7/10 (Standard). Style: flat design.

Role	Hex
Primary	#334155
Secondary	#475569
Accent / CTA	#059669
Background	#F8FAFC
Foreground	#0F172A
Card	#FFFFFF
Muted	#F2F3F4
Border	#E6E8EA
Destructive	#DC2626

Typography: Figtree for headings, Noto Sans for body. Spacing tokens from 4px (xs) to 64px (3xl).
Transitions 150–200 ms ease.

Anti-patterns: no emojis as icons, no complex shadows or 3D effects, cursor pointer on all clickable elements, no layout-shifting hovers, minimum 4.5:1 contrast, visible focus states, prefers-reduced-motion respected, responsive at 375/768/1024/1440 px.

11. Testing and Quality

- Vitest with jsdom covers domain logic: sales cart, pagination, units, barcode values and capture, stock alerts, activity grouping, reporting, permissions, rate-limit keys, validation schemas, receiving selection, environment parsing, and inventory data.

- ESLint (eslint-config-next) runs over the repository.
 - TypeScript strict typing across the app.
 - Next.js production build must pass before delivery.
 - Database tests run through supabase test db against the local stack.
-

12. Documentation Index

File	Contents
PRODUCT.md	Product definition, capabilities, constraints, principles
req.md	Project requirements
docs/page-guide.md	User-facing views and navigation flows
docs/endpoint-security.md	Verification rules, rate limits, deployment requirements
docs/supabase-setup.md	Supabase provisioning, migrations, seed, first admin
design-system/inventory-system/MASTER.md	Design tokens and UI rules
progress.md, decisions.md, results.md	Working logs (state summaries at the top of each)
rules.md	AI assistant working rules

13. Evidence and Limitations

The client supplied a low-resolution South Emerald Supermarket raster logo; approved SVG reconstructions live in public/brand/. No real customer records, commercial claims, testimonials, or production inventory data are available — the repository seed is development-only and clearly identified as synthetic.